TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

LABORATORY REPORT NO. 1

Caesar Cipher over the Romanian Alphabet

Cryptography: standard and keyword-permuted variants

Student: Gafenco Victor

Group: FAF-241

Coordinator: asist. univ. Zaica Maia

Chișinău - 2026

CONTENTS

Abstract **3**

Introduction **4**

1 Short theory **5**

1.1 Standard Caesar cipher 5

1.2 Keyword permutation 5

2 Implementation **6**

2.1 Letter encoding and normalization 6

2.2 Building the permutation 6

2.3 Encryption and decryption 6

2.4 Input validation and project structure 7

3 Results **8**

3.1 Encryption and decryption 8

3.2 Invalid input 10

3.3 Automated verification 10

Conclusions **11**

Source code **12**

ABSTRACT

This laboratory implements the Caesar cipher and its keyword-based extension in Java for the 31-letter Romanian alphabet. Both variants use explicit alphabet positions, accept shifts from 1 to 30, and provide encryption and decryption through an interactive console. The implementation validates text and keys, normalizes case and Romanian diacritics, and prints the keyword permutation. Console captures demonstrate successful round trips and recovery from invalid input. Automated tests cover every letter and permitted shift. The experiment illustrates the weakness of a small key space and the remaining frequency-analysis vulnerability of a monoalphabetic substitution.

Keywords: Caesar cipher, Romanian alphabet, permutation, Java, input validation

INTRODUCTION

The objective is to implement Tasks 1.1 and 1.2 of the supplied laboratory handout [**?**], using its Romanian letter encoding rather than ASCII or Unicode character values for shifts. A Maven parent project contains the `lab1` module, allowing later laboratories to be added independently. Java 17 is the minimum supported version.

1 SHORT THEORY

1.1 Standard Caesar cipher

The alphabet contains $n = 31$ letters, in the following order:

AĂÂBCDEFGHIÎJKLMNOPQRSȘTȚUVWXYZ.

The positions are $A = 0$, $\breve{A} = 1$, $\hat{A} = 2$, through $Z = 30$. For $k \in \{1, \ldots, 30\}$, encryption and decryption are:

$$c = (x + k) \bmod 31, \qquad m = (y - k) \bmod 31. \tag{1.1}$$

Here x and y are positions in the active alphabet. The standard cipher has 30 possible keys and can be attacked by trying every shift. Uppercasing and space removal mean decryption recovers the normalized plaintext, not its original formatting.

1.2 Keyword permutation

A keyword of at least seven Romanian letters determines a new alphabet. Its distinct letters are written in their first-occurrence order; unused letters follow in their natural order. The same formulas then use positions in this permuted alphabet for both input and output. For CRYPTOGRAPHY, the Romanian permutation is:

CRYPTOGAHĂÂBDEFIÎJKLMNQSȘȚUVWXZ.

Keyword length is checked before duplicate removal, so a seven-letter keyword with repeated letters is valid. Because arbitrary sequences are permitted, all $31!$ permutations are attainable. The nominal permutation–shift key space is $30 \cdot 31! \approx 2.47 \cdot 10^{35}$, but different pairs can produce the same substitution. In fact, conjugating a nonzero shift modulo the prime 31 yields a single 31-cycle, so this construction has $30!$ distinct substitutions. The larger nominal key space does not eliminate frequency analysis: a letter always maps to the same letter [**?**].

2 IMPLEMENTATION

2.1 Letter encoding and normalization

The constant `ALPHABET` stores exactly the handout's order. Lookup with `indexOf` obtains a position; `charAt` retrieves a letter. Unicode is used to represent and normalize letters, never to calculate the cipher shift. NFC normalization accepts decomposed diacritics; cedilla forms Ş and Ţ are mapped to Ș and Ț. Only ordinary spaces are removed from messages; keywords cannot contain spaces. Blank messages are rejected to keep the console interaction meaningful.

```
String composed = Normalizer.normalize(text, Normalizer.Form.NFC);
String letter = original.toUpperCase(Locale.ROOT)
        .replace('Ş', 'Ș').replace('Ţ', 'Ț');
String allowed = ALPHABET + ALPHABET.toLowerCase(Locale.ROOT) + "ŞşŢţ";
if (allowed.indexOf(original) < 0 || letter.length() != 1
        || ALPHABET.indexOf(letter.charAt(0)) < 0) {
    throw new IllegalArgumentException("Rejected character ...");
}
```

Each original code point is validated separately before concatenation. This prevents a non-Romanian character such as the German sharp s from being accepted through an uppercase expansion to two valid letters. Errors identify the rejected character and allowed alphabet.

2.2 Building the permutation

```
String normalized = normalize(keyword, false);
if (normalized.length() < 7) {
    throw new IllegalArgumentException("Keyword must contain at least 7 Romanian
   letters; spaces are not allowed.");
}
StringBuilder permutation = new StringBuilder();
for (char letter : (normalized + ALPHABET).toCharArray()) {
    if (permutation.indexOf(String.valueOf(letter)) < 0) {
        permutation.append(letter);
    }
}
```

The first occurrence of each letter is appended once. Iterating over the keyword followed by the natural alphabet guarantees all 31 letters appear. The console prints the completed permutation before processing the message.

2.3 Encryption and decryption

```
validateKey(key);
String normalized = normalize(text, true);
StringBuilder result = new StringBuilder();
for (char letter : normalized.toCharArray()) {
    int position = alphabet.indexOf(letter);
    int shifted = Math.floorMod(position + (decrypt ? -key : key),
```

```
                                    alphabet.length());
    result.append(alphabet.charAt(shifted));
}
```

Both operations share one implementation; only the direction differs. `Math.floorMod` handles negative subtraction and wraparound. The numeric key is validated before transformation.

2.4 Input validation and project structure

The console asks for the variant, operation, numeric key, optional keyword, and message. It retries the same field after an invalid entry. A generic input helper catches validation exceptions:

```
while (true) {
    System.out.print(prompt);
    if (!input.hasNextLine()) { return null; }
    try {
        return parser.apply(input.nextLine());
    } catch (IllegalArgumentException exception) {
        System.out.println("Invalid input: " + exception.getMessage());
    }
}
```

End of input terminates cleanly. The menu supports repeated operations and exit with zero. The root Maven POM aggregates `lab1`; the module builds an executable JAR. JUnit is a test dependency only, and the application has no external runtime dependencies.

3 RESULTS

3.1 Encryption and decryption

The images below render actual console sessions captured from the executable Java program through a pseudo-terminal, including echoed input. They can be regenerated using the capture script in the repository; adjacent text files preserve the raw transcripts. These are reproducible console captures, rather than desktop screenshots.

Figure 3.1 demonstrates standard encryption with $k = 3$ and subsequent decryption. The normalized message `CIFRULCEZAR` becomes `FKITXOFHÂBT`. The handout's English example uses a different alphabet and therefore gives a different ciphertext.

```
Laboratory 1 — Romanian Caesar cipher
Cipher (1 = standard, 2 = keyword, 0 = exit): 1
Operation (1 = encrypt, 2 = decrypt): 1
Numeric key (1–30): 3
Text: cifrul cezar
Ciphertext: FKITXOFHÂBT

Cipher (1 = standard, 2 = keyword, 0 = exit): 1
Operation (1 = encrypt, 2 = decrypt): 2
Numeric key (1–30): 3
Text: FKITXOFHÂBT
Plaintext: CIFRULCEZAR

Cipher (1 = standard, 2 = keyword, 0 = exit): 0
```

Figure 3.1 – Standard Romanian Caesar encryption and decryption

Figure 3.2 uses keyword `CRYPTOGRAPHY` and $k = 3$. The ciphertext is `PKJTXQPÎYÂT`; decryption with the same two keys recovers `CIFRULCEZAR`.

```
Laboratory 1 — Romanian Caesar cipher
Cipher (1 = standard, 2 = keyword, 0 = exit): 2
Operation (1 = encrypt, 2 = decrypt): 1
Numeric key (1–30): 3
Keyword (at least 7 Romanian letters, no spaces): cryptography
Permuted alphabet: CRYPTOGAHĂÂBDEFIÎJKLMNQSȘȚUVWXZ
Text: cifrul cezar
Ciphertext: PKJTXQPÎYÂT

Cipher (1 = standard, 2 = keyword, 0 = exit): 2
Operation (1 = encrypt, 2 = decrypt): 2
Numeric key (1–30): 3
Keyword (at least 7 Romanian letters, no spaces): cryptography
Permuted alphabet: CRYPTOGAHĂÂBDEFIÎJKLMNQSȘȚUVWXZ
Text: PKJTXQPÎYÂT
Plaintext: CIFRULCEZAR

Cipher (1 = standard, 2 = keyword, 0 = exit): 0
```

Figure 3.2 – Keyword permutation, encryption and decryption

3.2 Invalid input

Figure 3.3 shows rejection of zero, 31, and a noninteger numeric key, a six-letter keyword, a keyword containing a space, and a message containing an exclamation mark. Each rejected field is requested again. The corrected inputs complete encryption successfully.

```
Laboratory 1 — Romanian Caesar cipher
Cipher (1 = standard, 2 = keyword, 0 = exit): 2
Operation (1 = encrypt, 2 = decrypt): 1
Numeric key (1–30): 0
Invalid input: Key must be an integer from 1 to 30 inclusive.
Numeric key (1–30): 31
Invalid input: Key must be an integer from 1 to 30 inclusive.
Numeric key (1–30): abc
Invalid input: Key must be an integer from 1 to 30 inclusive.
Numeric key (1–30): 3
Keyword (at least 7 Romanian letters, no spaces): secret
Invalid input: Keyword must contain at least 7 Romanian letters; spaces are not allowed.
Keyword (at least 7 Romanian letters, no spaces): abc defg
Invalid input: Rejected character ' ' (U+0020). Allowed letters: AĂÂBCDEFGHIÎJKLMNOPQRSȘTȚUVWXYZ
(upper/lowercase, cedilla variants accepted); no spaces in the keyword.
Keyword (at least 7 Romanian letters, no spaces): cryptography
Permuted alphabet: CRYPTOGAHĂÂBDEFIÎJKLMNQSȘȚUVWXZ
Text: Salut!
Invalid input: Rejected character '!' (U+0021). Allowed letters: AĂÂBCDEFGHIÎJKLMNOPQRSȘTȚUVWXYZ
(upper/lowercase, cedilla variants accepted); spaces are removed.
Text: salut
Ciphertext: UÂQXA

Cipher (1 = standard, 2 = keyword, 0 = exit): 0
```

Figure 3.3 – Validation messages and successful recovery

3.3 Automated verification

Five JUnit tests check the exact alphabet, wraparound, known Romanian ciphertexts, the exact keyword permutation, case and space normalization, cedilla and decomposed diacritics, invalid numeric keys, forbidden characters and invalid keywords. The round-trip test covers every alphabet letter at all 30 shifts for both the natural alphabet and a keyword permutation, giving 60 full-alphabet round trips. The Maven `package` lifecycle runs these tests and builds the executable JAR.

CONCLUSIONS

The implementation demonstrates explicit Romanian letter encoding, modular arithmetic and the construction of a keyword-derived permutation. Encryption and decryption recover the normalized input for both tasks, while validation prevents malformed keys and unsupported characters from reaching the transformation. The standard cipher is insecure because only 30 shifts need to be tried. The keyword variant increases the possible substitutions but remains monoalphabetic and exposes repeated-letter patterns and language frequencies. Neither variant is appropriate for protecting real confidential data.

SOURCE CODE

The full Java source, Maven configuration, tests and report sources are maintained in the laboratory repository: `https://github.com/yoda-dinmd/crypt-labs`.